

CSE 315

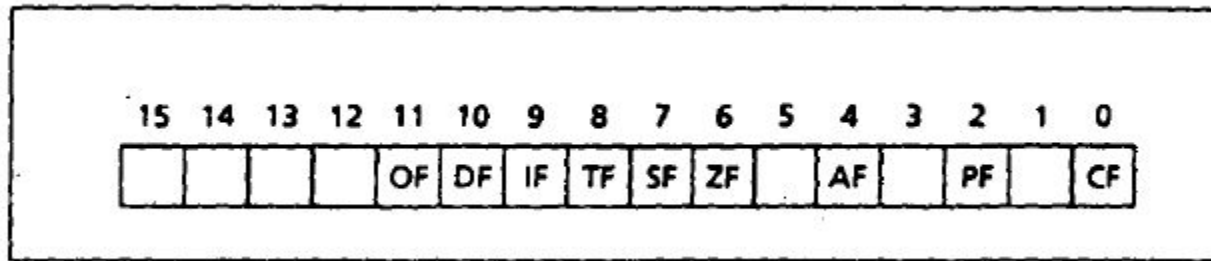
Microprocessors, Microcontrollers, and  
Embedded Systems

# Assembly Language: Flow Control

# Outline

- The Status Flags
- Control Flow Structure
  - Conditional Jump
  - Unconditional Jump
- Control Flow Structures
  - IF-THEN
  - IF-THEN-ELSE
  - CASE
- Branches with Compound Conditions
- Looping

# The FLAGS Register



# The FLAGS Register flags

| Flag            | Intel Mnemonic |
|-----------------|----------------|
| Overflow        | OF             |
| Sign            | SF             |
| Zero            | ZF             |
| Auxiliary Carry | AF             |
| Parity          | PF             |
| Carry           | CF             |

# Overflow Flag

- Unsigned Overflow (CF)

```
      1111 1111 1111 1111
+   0000 0000 0000 0001
-----
1 0000 0000 0000 0000
```

# Overflow Flag

- Signed Overflow (OF)

$$\begin{array}{r} 0111\ 1111\ 1111\ 1111 \\ +\ 0111\ 1111\ 1111\ 1111 \\ \hline 1111\ 1111\ 1111\ 1110 = \text{FFEH} \end{array}$$

# OF & CF

- $OF = 1$  for signed overflow,  $CF = 1$  for unsigned overflow.
- $CF = 1$ , if there is a carry out of the msb on addition.

However,  $CF$  is not affected by  $INC$  or  $DEC$ .

- $OF = 1$ , if we add two numbers of the same sign and the result is of different sign. If we add two numbers of the different sign, there is no way of overflow.
  - If the carry into and out of msb don't match  $OF = 1$

# ZF, SF and PF

- ZF = 1 if the result is zero.
- SF = 1, if the msb of result is 1.
- PF = 1, if the *low byte* of the result has an even number of 1s (even parity).



# Effect of the flags

**Example 5.3** SUB AX,BX, where AX contains 8000h and BX contains 0001h.

**Solution:**

$$\begin{array}{r} 8000h \\ - 0001h \\ \hline 7FFFh = 0111\ 1111\ 1111\ 1111 \end{array}$$

The result stored in AX is 7FFFh.

SF = 0 because the msb is 0.

PF = 1 because there are 8 (even number) one bits in the low byte of the result.

ZF = 0 because the result is nonzero.

CF = 0 because a smaller unsigned number is being subtracted from a larger one.

Now for OF. In a signed sense, we are subtracting a positive number from a negative one, which is like adding two negatives. Because the result is positive (the wrong sign), OF = 1.

# Outline

- The Status Flags
- Control Flow Structure
  - Conditional Jump
  - Unconditional Jump
- Control Flow Structures
  - IF-THEN
  - IF-THEN-ELSE
  - CASE
- Branches with Compound Conditions
- Looping

# An Example of Jump

## Display the entire IBM character set

```
.MODEL SMALL
.CODE

MOV     AH, 2          ; display char function
MOV     CX, 256        ; no. of chars to display
MOV     DL, 0          ; DL has ASCII code for null char

PRINT_LOOP:
INT     21H            ; display a char
INC     DL              ; increment ASCII code
DEC     CX              ; decrement counter
JNZ     PRINT_LOOP     ; keep going if CX not 0
.EXIT
END
```

# Conditional Jumps

- JNZ is an example of conditional jump instruction
  - Checks the Z flag. If  $Z = 0$  then jump to the location
- Three categories of conditional jumps
  - Signed jumps, for signed interpretation
  - Unsigned jumps, for unsigned interpretation
  - Single-flag jumps, operates on settings of individual flags

# The CMP Instruction

- The jump condition is often preceded by the CMP (compare) instruction

CMP destination, source

- It is like SUB, except that destination is not changed
- Destination may not be a constant
- The result is not stored but the flags are affected

```
CMP  AX, BX
JG   BELOW
```

If  $AX = 7FFFh$ , and  $BX = 0001h$ , the result is  $7FFFh - 0001h = 7FFEh$ .  
 $ZF = 0$ ,  $SF = OF = 0$ , JG (jump if greater) is satisfied,  
so control transfers to label BELOW

# Signed Conditional Jumps

| Symbol  | Description                                               | Condition for jumps |
|---------|-----------------------------------------------------------|---------------------|
| JG/JNLE | Jump if greater than<br>Jump if not less than or equal to | ZF = 0 and SF = OF  |
| JGE/JNL | Jump if greater than or equal to<br>Jump if not less than | SF = OF             |
| JL/JNGE | Jump if less than<br>Jump if not greater than or equal to | SF <> OF            |
| JLE/JNG | Jump if less than or equal<br>Jump if not greater than    | ZF = 1 or SF <> OF  |

# Unsigned Conditional Jumps

| Symbol  | Description                                 | Condition for jumps |
|---------|---------------------------------------------|---------------------|
| JA/JNBE | Jump if above<br>Jump if not below or equal | CF = 0 and ZF = 0   |
| JAЕ/JNB | Jump if above or equal<br>Jump if not below | CF = 0              |
| JB/JNAE | Jump if below<br>Jump if not above or equal | CF = 1              |
| JBE/JNA | Jump if below or equal<br>Jump if not above | CF = 1 or ZF = 1    |

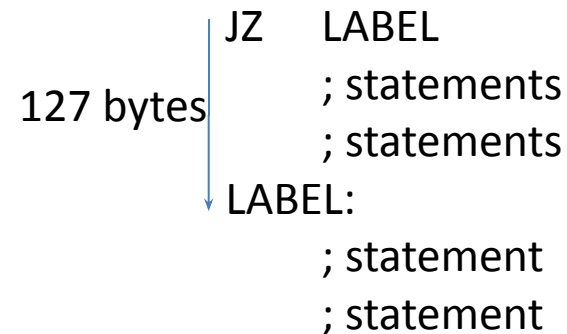
# Single-Flag Jumps

| Symbol  | Description                            | Condition for jumps |
|---------|----------------------------------------|---------------------|
| JE/JZ   | Jump if equal<br>Jump if equal to zero | ZF = 1              |
| JNE/JNZ | Jump if not equal<br>Jump if not zero  | ZF = 0              |
| JC      | Jump if carry                          | CF = 1              |
| JNC     | Jump if no carry                       | CF = 0              |
| JO      | Jump if overflow                       | OF = 1              |
| JNO     | Jump if no overflow                    | OF = 0              |
| JS      | Jump if sign negative                  | SF = 1              |
| JNS     | Jump if nonnegative sign               | SF = 0              |
| JP/JPE  | Jump if parity even                    | PF = 1              |
| JNP/JPO | Jump if parity odd                     | PF = 0              |



# Range of a Conditional Jump

- The destination label must precede the jump instruction by no more than 126 bytes
- Or, follow by no more than 127 bytes



# Signed vs. Unsigned Jumps

- Each signed jump corresponds to an analogous unsigned jump
  - e.g. signed JG corresponds to unsigned JA
  - Use depends on the interpretation
- The jumps operate on different flags

| Symbol  | Description                                               | Condition for jumps |
|---------|-----------------------------------------------------------|---------------------|
| JG/JNLE | Jump if greater than<br>Jump if not less than or equal to | ZF = 0 and SF = OF  |
| JA/JNBE | Jump if above<br>Jump if not below or equal               | CF = 0 and ZF = 0   |

# Signed vs. Unsigned Jumps cont.

- For signed interpretation, let us take
  - AX = 7FFFh, BX = 8000h and we execute

|     |        |
|-----|--------|
| CMP | AX, BX |
| JA  | BELOW  |

- Then, even though 7FFFh > 8000h in a signed sense, the program does not jump to BELOW
- Because 7FFFh < 8000h in an unsigned sense
- We used JA, which is the unsigned jump

# The JMP Instruction

- JMP (jump) instruction causes an unconditional jump
- The syntax is: `JMP destination`
- JMP can be used to get around the range restriction

TOP:

; body of the loop, say 2 instructions

DEC CX ; decrement counter

JNZ TOP ; keep looping if CX > 0

MOV AX, BX

TOP:

; body of the loop contains many instructions

DEC CX

JNZ BOTTOM

JMP EXIT

BOTTOM:

JMP TOP

EXIT:

MOV AX, BX

# Outline

- Control Flow Structure
  - Conditional Jump
  - Unconditional Jump
- Control Flow Structures
  - IF-THEN
  - IF-THEN-ELSE
  - CASE
- Branches with Compound Conditions
- Looping

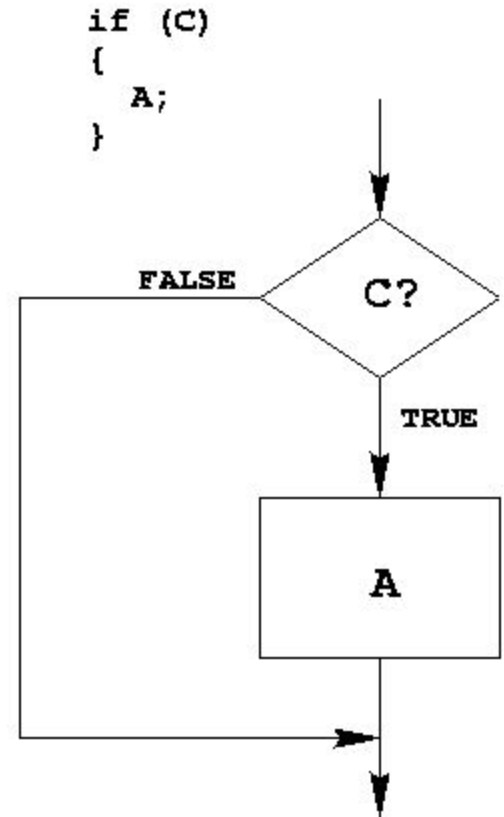
# IF-THEN Structure

Replace the number in AX by its absolute value.

```
IF AX < 0 THEN  
    replace AX by -AX  
END_IF
```

```
    CMP    AX, 0    ; AX < 0?  
    JNL   END_IF  
    NEG AX  
END_IF:
```

**Example 6-2: Assembly Language Programming**



# IF-THEN-ELSE Structure

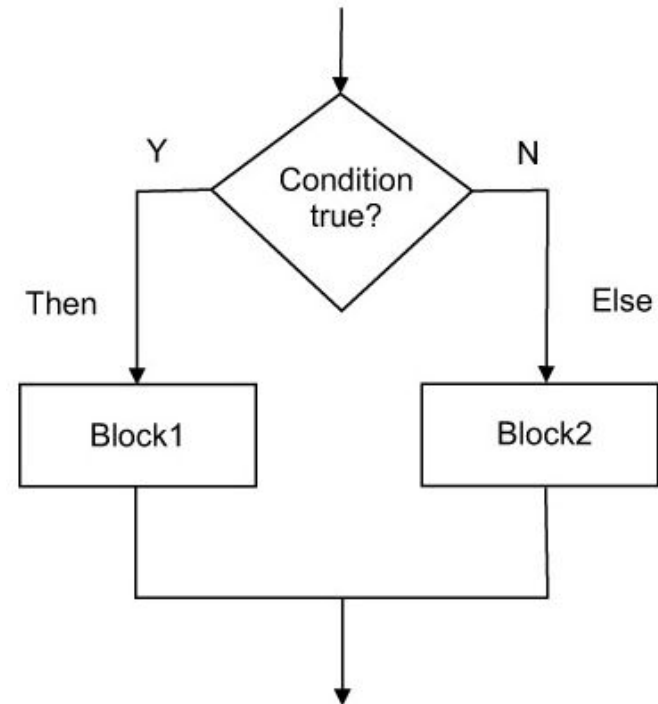
Suppose AL and BL contains ASCII characters.  
Display the one that comes first in the character sequence

```
IF AL <= BL THEN
    display the character in AL
ELSE
    display the character in BL
END_IF
```

```
MOV AH, 2; prepare to display
CMP AL, BL    ; AL <= BL?
JNBE ELSE_

MOV     DL, AL
JMP  DISPLAY

ELSE_:
    MOV     DL, BL
DISPLAY:
    INT  21h
END_IF:
```



# CASE

- A CASE is a multi-way branch structure

CASE expression

1: statements\_1

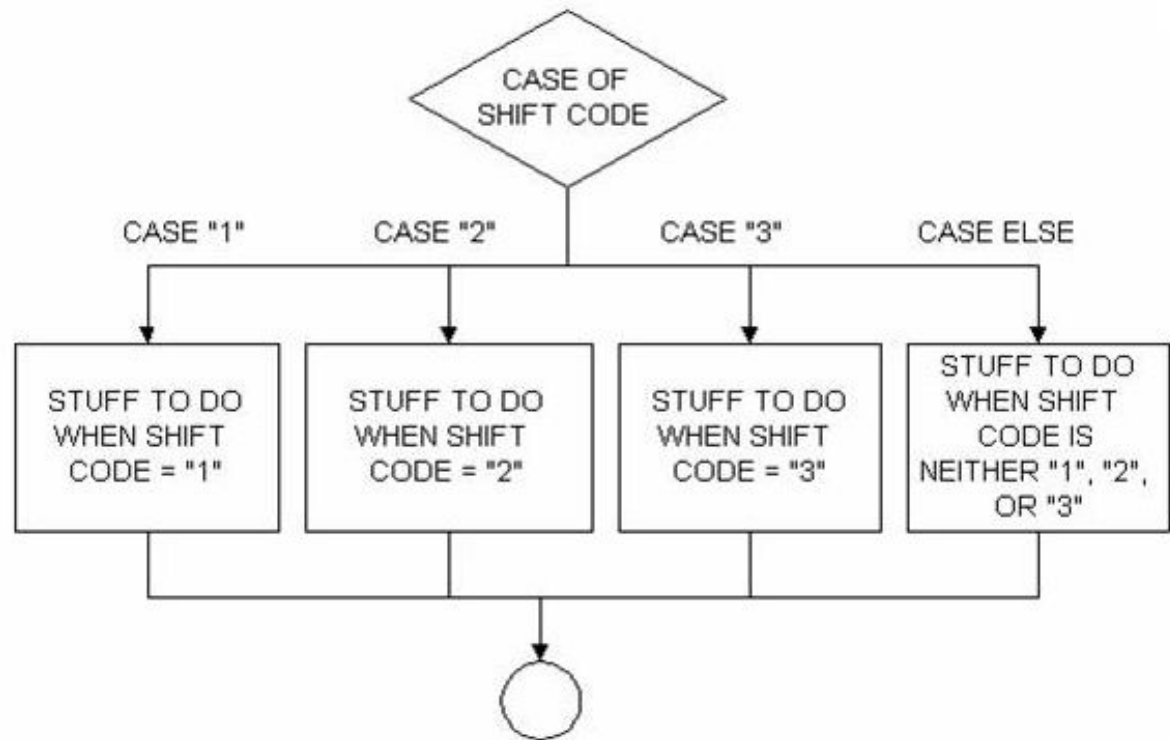
2: statements\_2

\*

\*

n: statements\_n

END\_CASE





# CASE Example

If AX contains a negative number, put -1 in BX;

If AX contains 0, put 0 in BX;

If AX contains a positive number, put 1 in BX.

```
    CMP     AX, 0           ; test AX
    JL      NEGATIVE       ; AX < 0
    JE      ZERO           ; AX = 0
    JG      POSITIVE       ; AX > 0
NEGATIVE:
    MOV     BX, -1         ; put -1 in BX
    JMP     END_CASE       ; and exit
ZERO:
    MOV     BX, 0          ; put 0 in BX
    JMP     END_CASE       ; and exit
POSITIVE:
    MOV     BX, 1          ; put 1 in BX
END_CASE:
```

```
CASE  AX
      < 0: put -1 in BX
      = 0: put 0 in BX
      > 0: put 1 in BX
END_CASE
```

**Example 6-4: Assembly Language Programming**

# More CASE Example

If AL contains 1 or 3, display “o” for odd;  
If AL contains 2 or 4, display “e” for even;

```
CMP AL, 1 ; AL = 1?
JE  ODD ; yes, display 'o'
CMP AL, 3 ; AL = 3?
JE  ODD ; yes, display 'o'
CMP AL, 2 ; AL = 2?
JE  EVEN ; yes, display 'e'
CMP AL, 4 ; AL = 4?
JE  EVEN ; yes, display 'e'
JMP END_CASE

ODD:
    MOV     DL, 'o'      ; get 'o'
    JMP    DISPLAY      ; go to display

EVEN:
    MOV DL, 'e'      ; get 'e'

DISPLAY:
    MOV AH, 2 ; char display function
    INT  21h ; display character
END_CASE
```

```
CASE  AL
      1, 3: display 'o'
      2, 4: display 'e'
END_CASE
```

# Outline

- Control Flow Structure
  - Conditional Jump
  - Unconditional Jump
- Control Flow Structures
  - IF-THEN
  - IF-THEN-ELSE
  - CASE
- Branches with Compound Conditions
- Looping

# Branches with Compound Conditions

- Branching condition in an IF or CASE can be

condition\_1    AND    condition\_2  
condition\_1    OR    condition\_2

- First one is **AND condition**
- Second one is **OR condition**

# AND Conditions

Read a character, and if it's an uppercase letter, display it.

```
Read a character into AL
IF ('A' <= character ) and (character <= 'Z') THEN
    display the character
END_IF
```

```
MOV     AH, 1      ; read character function
INT 21h ; char in AL
```

```
CMP AL, 'A'      ; char >= 'A'
JNGE     END_IF  ; no, exit
CMP AL, 'Z'      ; char <= 'Z'
JNLE     END_IF  ; no, exit
```

```
MOV     DL, AL    ; get char
MOV     AH, 2     ; display character function
INT 21h ; display the character
END_IF:
```

**Example 6-6: Assembly Language Programming**

# OR Conditions

Read a character, and if it's 'y' or 'Y', display it; otherwise, terminate the program

```
Read a character into AL
IF (character = 'y') or (character = 'Y') THEN
    display the character
ELSE
    terminate the program
END_IF
```

```
MOV     AH, 1; read character function
INT     21h ; char in AL

CMP     AL, 'Y' ; char = 'Y'
JE      THEN; yes, display the char
CMP     AL, 'y' ; char = 'y'
JE      THEN; yes, display the char
JMP     ELSE_

THEN:
MOV     DL, AL ; get the char
MOV     AH, 2; display character function
INT     21h ; display the character
ELSE_:
```

**Example 6-7: Assembly Language Programming**

# Looping

# Outline

- Control Flow Structures
  - FOR Loop
  - WHILE Loop
  - REPEAT-UNTIL Loop
- Programming with Higher Level Structures



# FOR Loop

Write a program to display a row of 80 stars '\*'

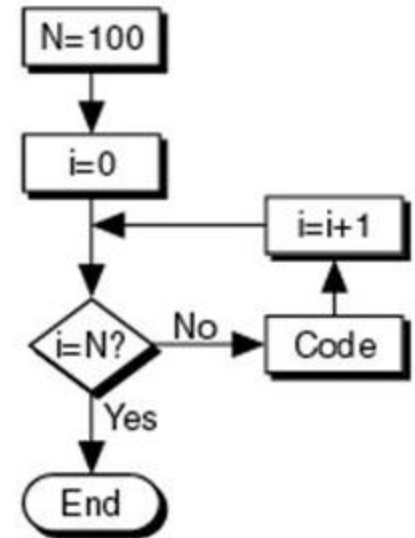
```
FOR 80 times DO  
    display '*'  
END_FOR
```

```
MOV    CX, 80    ; number of '*' to display  
MOV    AH, 2     ; char display function  
MOV    DL, '*'   ; char to display
```

TOP:

```
INT 21h ; display a star  
LOOP TOP ; repeat 80 times
```

**Example 6-8: Assembly Language Programming**



# Caution!

- 'For loop' implemented using LOOP is executed at least once
- If CX contains 0 when loop is entered, the LOOP instruction will decrement CX to FFFFh
- The loop will be executed 65535 more times
- JCXZ (jump if CX is zero) may be used

```
JCXZ SKIP
TOP:
    ; body of the loop

    LOOP    TOP
SKIP:
```

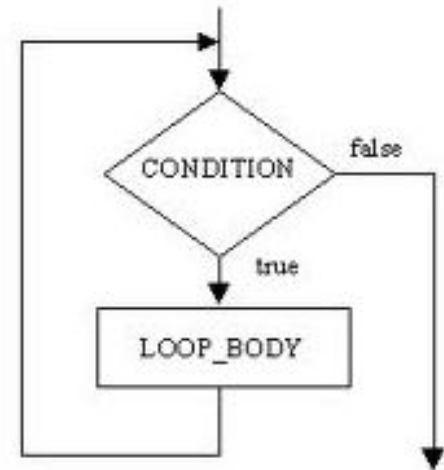
# WHILE Loop

Write a program to count the characters in an input line

```
Initialize count to 0
Read a character
WHILE character <> carriage_return DO
    count = count + 1
    read a character
END_WHILE
```

```
MOV     DX, 0      ; DX counts the characters
MOV     AH, 1      ; read char function
INT     21h ; read a char in AL
WHILE_:
CMP     AL, 0DH    ; CR?
JE      END_WHILE
INC     DX
INT     21h
JMP     WHILE_
END_WHILE:
```

```
WHILE condition DO
    statements
END_WHILE
```



**Example 6-9: Assembly Language Programming**

# REPEAT Loop

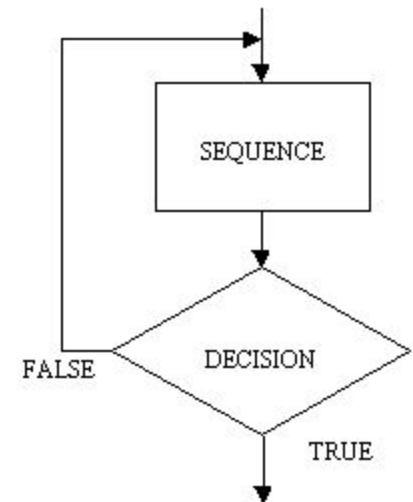
Write a program to read characters until a blank/space is read

```
REPEAT
    read a character
UNTIL character is a blank
```

```
MOV     AH, 1      ; read char function
REPEAT:
    INT  21h      ; read a char in AL
    CMP   AL, ' '  ; a blank?
    JNE  REPEAT    ; no, keep reading
```

**Example 6-10: Assembly Language Programming**

```
REPEAT
    statements
UNTIL condition
```



# References

- Ch 5, 6 Assembly Language Programming – by Yu and Marut